

Sistemas Embebidos E1504

Ejercicio Especial 1 – Medidor R/C con Autorango

Santiago Barbero y Jeremías Picardo

1. Introducción.

El presente trabajo práctico consiste en el desarrollo de un medidor de resistencia y capacitancia con autorango utilizando un microcontrolador STM32 y el toolchain que ofrece STM32.

El sistema implementa una interfaz UART para interactuar con el usuario mediante un menú de configuración, permitiendo seleccionar el tipo de medición y el modo de funcionamiento.

Para la implementación se utilizaron periféricos como los GPIO, el Sysctik, el ADC todos dentro del microcontrolador, además de una máquina de estados como método principal de organización del software.

El objetivo principal del trabajo fue aplicar conceptos de programación de sistemas embebidos, control de periféricos y organización modular del código.

2. Objetivos.

- Implementar una máquina de estados para controlar el funcionamiento del sistema.
- Realizar mediciones de resistencia y capacitancia utilizando ADC y GPIO.
- Implementar autorango para medición de resistencia.
- Desarrollar una interfaz UART con menú interactivo.
- Organizar el código utilizando librerías y funciones modulares.

3. Descripción general del sistema.

El usuario puede:

- Seleccionar el parámetro a medir (resistencia o capacitancia).
- Seleccionar el modo de medición (única o periódica).
- Iniciar o detener las mediciones mediante un pulsador.

El sistema muestra los resultados de las mediciones en formato ASCII a través de la terminal UART.

El hardware utilizado consiste en:

- Microcontrolador STM32F103
- Resistencias de:
 - 330 Ω
 - 10 k Ω
 - 1 M Ω
- Pulsador
- Puerto UART
- Entrada ADC
- DUT (Device Under Test), R o C
- LED (No era necesario es para ver si se recibe cuando se aprieta el pulsador)

Con el siguiente esquemático de conexión:

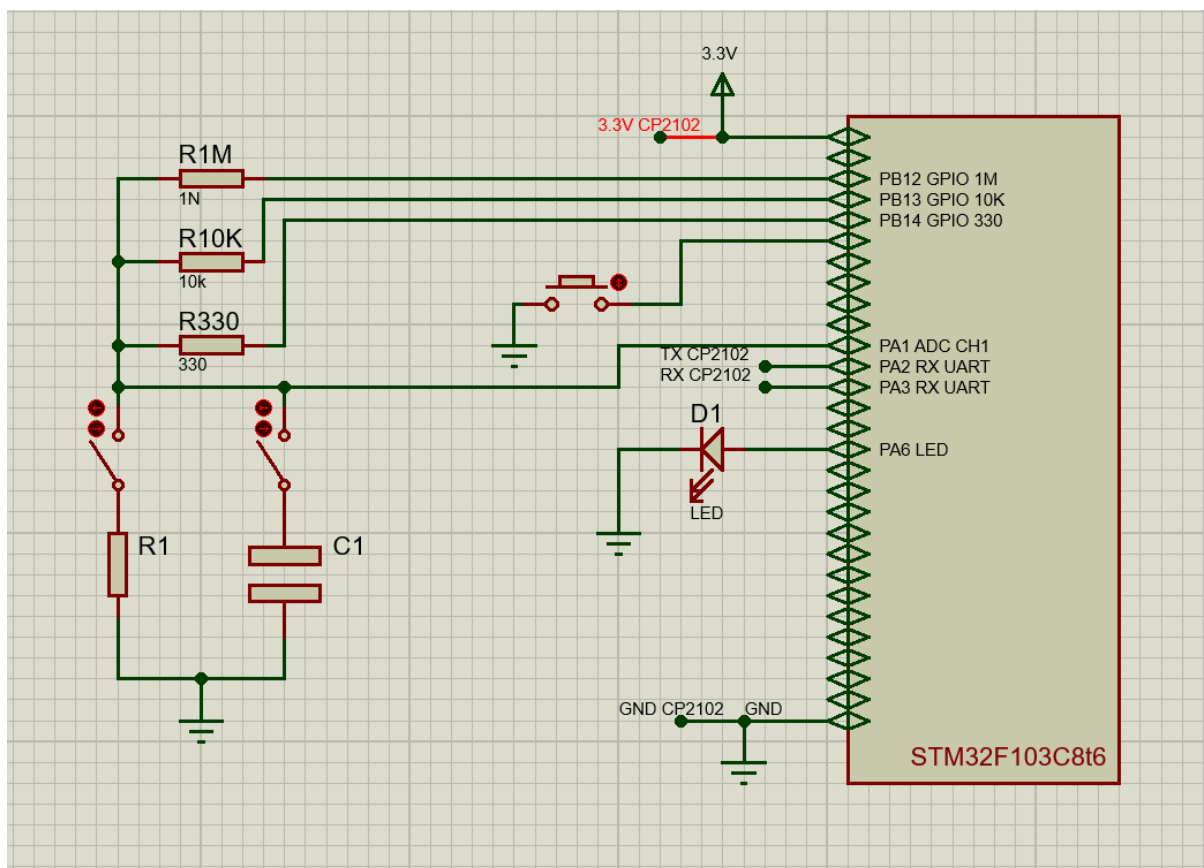


Figura 1: Esquemático

(Las llaves no son parte del circuito real, son para demostrar que el usuario puede elegir que medir).

5. Funcionamiento de la medición de resistencia

La medición de resistencia se realiza utilizando un divisor resistivo y el periférico ADC del microcontrolador.

Para implementar el autorango, se utilizan tres resistencias de referencia:

- 330 Ω
- 10 k Ω
- 1 M Ω

El sistema comienza utilizando la resistencia más baja. Si la tensión medida supera el 95% del rango del ADC, se selecciona automáticamente la siguiente resistencia de referencia. Una vez seleccionado el rango adecuado, se toma la medición y se calcula el valor de la resistencia del DUT mediante la ecuación del divisor resistivo.

6. Funcionamiento de la medición de capacitancia

La medición de capacitancia se implementó mediante el análisis temporal de la carga del capacitor.

Primero se descarga completamente el capacitor utilizando la resistencia de 330 Ω .

Luego, el capacitor se carga mediante la resistencia de 1 M Ω mientras el sistema monitorea periódicamente el valor leído por el ADC.

Cuando la tensión del capacitor alcanza aproximadamente el 63% del valor final, el tiempo transcurrido se utiliza para calcular la capacitancia.

El sistema incluye un timeout para evitar bloqueos en caso de que la medición no pueda completarse correctamente.

7. Desarrollo máquina de estados y librería.

La máquina de estados fue implementada íntegramente utilizando una librería en el código bautizada como "menu". La misma ofrece dos funciones de alto nivel: menu_init y menu_procesarEvento.

La función de inicialización menu_init recibe 6 parámetros. El primero de ellos es un struct "medicion" definido en el header de la librería, que contiene todas las variables necesarias para el funcionamiento de la máquina de estados, principalmente el estado actual de la máquina, el modo y el parámetro de medición. La función recibe también los valores iniciales para el modo de medición y el parámetro, y los punteros a handlers para el canal UART, el ADC y el timer que utiliza la librería.

La función menu_procesarEvento se llama desde el main cada vez que ocurre un evento. Los mismos se manejan en el código desde el while(1), e incluyen:

- Eventos PULSADOR, cada vez que se detecta una nueva pulsación del botón.
- Eventos BOTON_1, cuando se recibe un carácter '1' desde la UART.
- Eventos BOTON_2, cuando se recibe un carácter '2' desde la UART.
- Eventos TIMER, cuando se produce una interrupción del timer. El timer es activado y desactivado desde la librería, y el tiempo de interrupción está configurado en el main desde el software CubeMX con un prescaler de 7200 y un Counting Period de 1000. Utilizando el reloj del kit a 72MHz, estos valores aseguran una interrupción cada 100ms.

Se presenta a continuación un diagrama de la máquina de estados, con los cambios entre estado que se producen por cada evento.

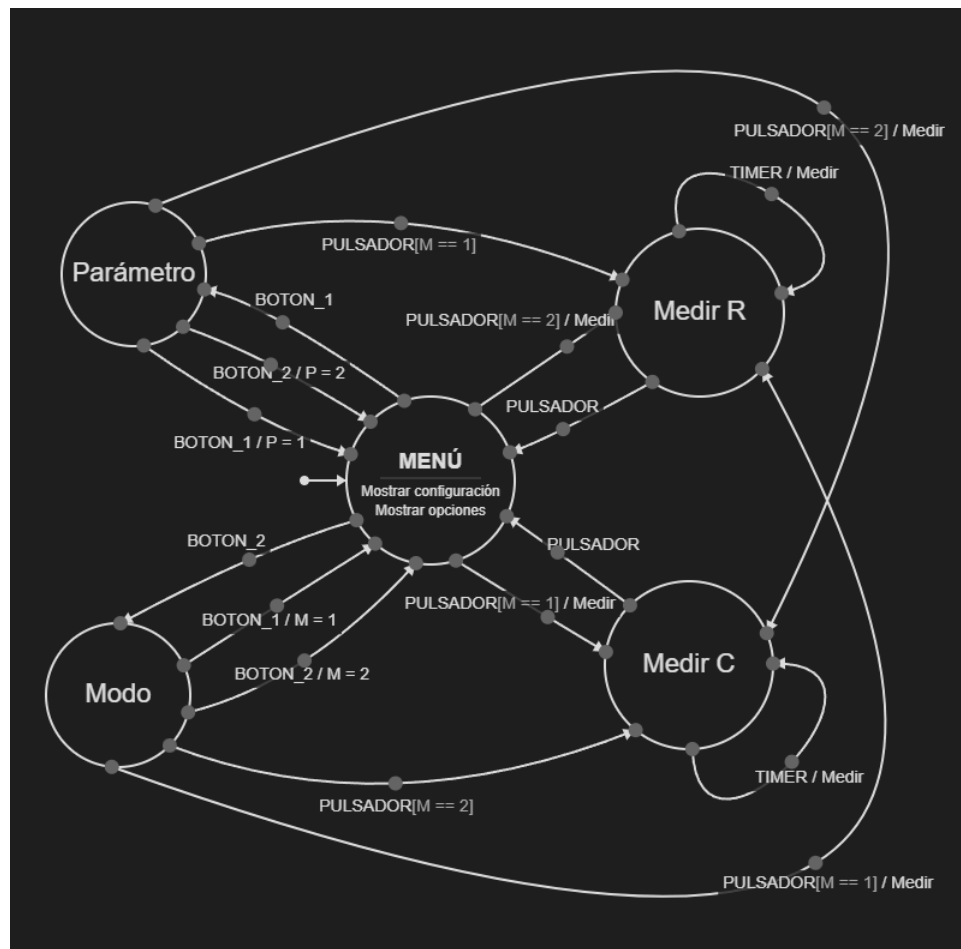


Figura 2: Máquina de estados finitos

8. Control de versiones con Git

En el inicio del proyecto se utilizó Git como sistema de control de versiones, aunque no se respetó tanto para evitar conflictos entre lo que cada integrante iba atribuyendo en el código, finalmente se decidió que uno solo realizaba los commits y push, para invitar interponerse.

A su vez se conservaba un respaldo de cada program con el progreso hasta el momento.

Link: https://github.com/SantiAB1/trabajo_especial_1/tree/mai

9. Resultados obtenidos

El sistema desarrollado permite realizar mediciones de resistencia y capacitancia mediante UART, con un error estimado del 5% para ambos parámetros.

La medición de resistencia funcionó correctamente utilizando autorango automático.

La medición de capacitancia permitió estimar el valor del capacitor a partir del tiempo de carga utilizando el ADC y GPIO del microcontrolador.

10. Conclusión

El trabajo permitió aplicar conceptos fundamentales de sistemas embebidos, incluyendo:

- uso de periféricos ADC y GPIO
- máquinas de estados
- modularización del software
- control temporal
- comunicación UART

Además, se adquirió experiencia práctica en el desarrollo y depuración de firmware sobre microcontroladores STM32.